Cryptosystem Implementation Project

CPSC 3730 · University of Lethbridge

Presenter: Nigel Noori · Student ID: 001237119 · Professor: Dr. Hua Li

Project Introduction & Goals

Objective

Build a fully functional cryptosystem integrating key generation, key exchange, encryption/decryption, and digital signatures — entirely from scratch.

Core Constraint

Built-in cryptographic functions and external libraries (e.g., `hashlib`) were strictly prohibited. Every algorithm was implemented manually.

Approach

Deep-level, manual implementation of each mathematical primitive to demonstrate full understanding of the underlying cryptographic principles.

Modular System Architecture

The project is organized into four focused Python modules, each with a single responsibility, all orchestrated through a central CLI.

`main.py`

Interactive CLI that ties all modules together, providing step-by-step mathematical outputs for each action.

`crypto_math.py`

Foundational library built from scratch: `gcd`, `mod_pow`, `is_prime`.

`rsa_system.py`

Complete RSA protocol: key generation, character-by-character encryption, and digital signatures.

`dh_exchange.py`

Models the Diffie-Hellman key exchange process between two participants (Alice and Bob).

RSA: Beyond Black Boxes

1 Key Generation

Prime numbers are generated using a manual Miller-Rabin primality test. The private key is computed via the Extended Euclidean Algorithm — no library shortcuts.

2 Encryption & Decryption

Standard modular exponentiation is applied to the ASCII value of each character individually, preventing integer overflow during terminal-based demonstration.

3 Digital Signatures

With `hashlib` forbidden, a custom polynomial rolling hash was implemented to produce a unique message digest, which is then signed and verified using RSA.

```
--> [RSA] Starting Key Generation
[SYS] Searching for a 8-bit prime number...
[SYS] Found prime!
[SYS] Searching for a 8-bit prime number...
[SYS] Found prime!
[RSA] p = 181, q = 223
[RSA] Modulus (n) = 40363, Totient (phi) = 39960
[RSA] Public Key: (e=7, n=40363)
[RSA] Private Key: (d=28543, n=40363)
.....................................
```

```
Enter a short message to encrypt: nigel098

[SYS]     Encryption Phase
[RSA] Encrypting message: 'nigel098'
      Char 'n' (ASCII 110) -> Encrypted: 354
      Char 'i' (ASCII 105) -> Encrypted: 38167
      Char 'g' (ASCII 103) -> Encrypted: 21567
      Char 'e' (ASCII 101) -> Encrypted: 8106
      Char 'l' (ASCII 108) -> Encrypted: 5568
      Char '0' (ASCII 48) -> Encrypted: 10727
      Char '9' (ASCII 57) -> Encrypted: 27589
      Char '8' (ASCII 56) -> Encrypted: 20756

[RESULT] Final Ciphertext Array: [354, 38167, 21567, 8106, 5568, 10727, 27589, 20756]

[SYS]     Decryption Phase
[RSA] Decrypting ciphertext: [354, 38167, 21567, 8106, 5568, 10727, 27589, 20756]
      Cipher 354 -> Decrypted ASCII 110 -> Char 'n'
      Cipher 38167 -> Decrypted ASCII 105 -> Char 'i'
      Cipher 21567 -> Decrypted ASCII 103 -> Char 'g'
      Cipher 8106 -> Decrypted ASCII 101 -> Char 'e'
      Cipher 5568 -> Decrypted ASCII 108 -> Char 'l'
      Cipher 10727 -> Decrypted ASCII 48 -> Char '0'
      Cipher 27589 -> Decrypted ASCII 57 -> Char '9'
      Cipher 20756 -> Decrypted ASCII 56 -> Char '8'

[RESULT] Final Decrypted Message: nigel098
```

Diffie-Hellman: Secure Key Sharing

Agree on p & g

Choose private keys

Compute & exchange

Derive shared secret

```
[SYS] Using public prime p = 23 and base g = 5

[Alice] Generated Private Key: 19
[Alice] Computed Public Key: 7
------------------------------
[Bob] Generated Private Key: 19
[Bob] Computed Public Key: 7

[SYS]     Exchange Phase
[Alice] Computing shared secret using received public key 7...
[Alice] Final Shared Secret: 11
[Bob] Computing shared secret using received public key 7...
[Bob] Final Shared Secret: 11

[SYS]     Verification
[SUCCESS] Both parties share the secret key: 11
```

The `dh_exchange.py` module simulates two users, Alice and Bob, who generate random secret private keys and compute public keys using shared public parameters (prime) and (base). Each participant then independently derives the identical shared secret, verifying the correctness of the exchange without ever transmitting the secret itself.

Live Demonstration

This section transitions to a live walkthrough of the completed cryptosystem in action.

📟 CLI Interface

Navigate the interactive menu in `main.py` and observe real-time mathematical output at each step.

🔑 RSA Demo

Generate key pairs, encrypt a message character-by-character, and verify a digital signature — all computed manually.

🔐 DH Exchange

Watch Alice and Bob establish a shared secret over an insecure channel, confirming identical results on both sides.